

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D054.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 054 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

Numba JIT 加速

// Numba

KEY CONCEPTS · 三大要点

- @jit
- nopython
- 并行

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面几节我们学了 NumPy 数组、pandas 表格,这一节解决一个很现实的痛点:用 python 写的数值循环太慢了。原因是 python 平时是边读边翻译地执行,每跑一行都现场翻译,跑几十万次循环自然慢。这一节请出 Numba,它的核心武器是一个叫 @njit 的装饰器,就像给你的函数请来一位翻译官,第一次调用时把整段函数一次性翻译成机器母语(也就是编译),之后再调用就直接跑机器码,常常能快上几十倍。代价有两个:一是它只认数字和 numpy 数组,不认 pandas 表格,所以要先把表转成纯数字数组;二是第一次调用要花点时间编译,从第二次起才是真实速度。我们用紫金矿业的真实日线,把同一个回测循环写成普通版和加了 @njit 的版本,亲眼看快慢差几十倍。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解 python 数值循环为什么慢:边读边翻译地执行,几十万次循环就拖死
- 02 会用 Numba 的 @njit 装饰器,只加一行就把函数编译成机器码,提速几十倍
- 03 搞懂 nopython 模式:这才是真正加速的模式,也是 @njit 默认的模式
- 04 知道 Numba 只认数字和 numpy 数组,不认 pandas,要先把表转成纯数组
- 05 会用 prange 加 parallel=True 让循环吃满多核 CPU,在加速之上再提速

02 · CONTEXT · 历史与背景

老聂改一个参数就得等几分钟,加一行装饰器后几秒钟跑完

// 不读历史的策略走不远

老聂是个会写 python 的散户,自己撸了一套回测,逐日遍历几年行情算均线信号、累计净值。代码逻辑没问题,就是慢得让人崩溃:跑一次要等好几分钟,他想试不同的均线参数,改一个数字就得从头再等一遍,一个下午折腾下来也试不了几组,人都熬麻了。后来他在一个量化群里看到有人提了一句 Numba,说在那个最耗时的循环函数头上加一行 @njit 装饰器就行,别的几乎不用改。老聂将信将疑地试了试,只在自己的回测函数上面加了一行,第一次跑还是要等一会(他后来才知道那一会是在编译),可第二次再跑,几秒钟就出结果,惊为天人。同样一套逻辑,同样一台电脑,就因为多了一个装饰器,慢得让他想砸键盘的几分钟变成了眨眼的几秒钟。从那以后他终于能一口气把上百组参数都跑一遍,而不是改一个等半天。他唯一踩过的坑是一开始直接把 pandas 表格喂给加了 @njit 的函数,程序报错,后来才明白 Numba 只认纯数字数组,先把那一列价格转成 numpy 数组就好了。

03 · CORE CONCEPTS · 核心概念详解

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

JIT 即时编译:第一次把函数翻译成机器母语

JIT 是即时编译的意思。打个比方:平时 python 跑代码像找了个同声传译,你说一句他翻一句,每一行都现场翻译,跑几十万次循环就累死了。JIT 不一样,它在你第一次调用某个函数时,把整段函数一次性翻译成电脑直接能懂的机器母语(机器码),翻译好之后存起来,以后再调这个函数就直接跑翻译好的机器码,不用再现场翻译,自然快得多。举个最小例子:一个累加一百万次的循环,边翻译边跑要几百毫秒,翻译成机器码后只要几毫秒。这就是 JIT 提速的根本道理。

CONCEPT_02

@njit:一个装饰器,你的函数几乎不用改

Numba 给你的工具特别省事,就是一个叫 @njit 的装饰器。用法是在你原来那个慢函数的上面单独加一行 @njit,函数体一个字都不用动。例如你有个 def backtest(prices): ... 的回测函数,只要在 def 上面加一行 @njit,它会在第一次被调用时自动编译成机器码。装饰器你可以理解成给函数贴了一张『请编译我』的标签,Numba 看到这张标签就接手。这是 Numba 最迷人的地方:几乎零改动,提速几十倍。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

nopython:这才是真正加速的模式

njit 里的 n 就是 nopython 的意思,指的是 no python object,完全不依赖 python 那套慢吞吞的对象机制,纯用机器码跑。这是 Numba 真正提速的模式,也是 @njit 默认就帮你开好的模式。打个比方:nopython 模式就像让运动员脱掉厚重的外套轻装上阵,跑得才快。如果你的函数里写了 Numba 不支持的东西,它可能会偷偷退回到带 python 的慢模式,那就几乎没有加速了,所以 @njit 默认强制 nopython,不支持就直接报错提醒你,而不是悄悄变慢。

CONCEPT_04

只认 numpy 不认 pandas:先把表转成纯数字数组

Numba 有个很重要的脾气:它只认数字和 numpy 数组,不认 pandas 的 DataFrame 表格。这是因为 pandas 表格内部结构复杂,翻译成机器码很困难。所以正确做法是:在把数据交给 @njit 函数之前,先把要用的那一列从表格里抽出来,转成纯数字的 numpy 数组。最小例子:你有一个价格表 df,要把收盘价喂给加速函数,先写 `prices = df['close'].to_numpy()`,拿到一条纯数字数组,再传进 @njit 函数,就不会报错了。记住:表格转数组,是用 Numba 前的固定动作。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

prange 加 parallel=True:让循环吃满多核

@njit 已经很快了,如果你的循环每一圈互相独立(比如同时回测100组不同参数,各算各的互不干扰),还能在加速之上再榨一层:用 prange 替代普通的 range,并在装饰器里写成

@njit(parallel=True),Numba 就会把这层循环自动分配到电脑的多个 CPU 核心上同时跑。打个比方:本来一个人挨个处理100件活,现在叫来八个人一人分十几件同时干,自然更快。注意一个铁律:prange 必须配 parallel=True 才会真正并行,光写 prange 不写 parallel=True 是没用的。

04 · PYTHON LAB · 真实可跑代码

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_054_numba_jit.py — Numba JIT 即时编译:把慢的 python 数值循环提速几十倍
# 真名上屏:Numba / @njit(nopython=True 真正加速)/ prange + parallel=True 多核并行 / 不
认 pandas 要先转 numpy 数组
# 核心比喻:平时 python 边读边翻译地跑很慢;@njit 像请翻译官第一次把整个函数翻译成机器母语(编
译),之后直接跑机器码快几十倍
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs
from numba import njit, prange

def _data_path(_name):
    # 铁律62:data/ 放在 notebook 文件夹里。仓库根(run_lab)存取 out/notebook/data/;
    # 原版 notebook 在 out/notebook/ 用 data/;中国版在 out/notebook/cn/ 用 ../data/
    from pathlib import Path as _P
    _here = _P.cwd()
    for _b in [_here/'data', _here/'../'/'data', _here/'out'/'notebook'/'data',
               _here/'../'/'../'/'data', _here/'../'/'../'/'../'/'data']:
        if (_b/_name).exists():
            return str(_b/_name)
    if (_here/'out'/'notebook').exists():
        _t = _here/'out'/'notebook'/'data'
```

```
elif _here.name == 'cn':  
    _t = _here/ '../data'  
else:
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
_t = _here/'data'
_t.mkdir(parents=True, exist_ok=True)
return str(_t/_name)

plt.rcParams['axes.unicode_minus'] = False
CODE, NAME = 'sh.601899', '紫金矿业'
START, END = '2018-01-01', '2024-12-31'
CSV_PATH = _data_path('D054_numba.csv')

# ==== 0. 拉紫金矿业日线收盘,转成纯 numpy 数组(Numba 不认 pandas,只认数字数组)====
print('==== 0. 拉紫金矿业日线收盘,转 numpy 数组 ====')
import os
if os.path.exists(CSV_PATH):
    # 缓存优先:CSV 在就直接读,不联网
    px = pd.read_csv(CSV_PATH, parse_dates=['date'])
else:
    lg = bs.login()
    if lg.error_code != '0':
        raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
    rs = bs.query_history_k_data_plus(CODE, 'date,close', start_date=START,
                                     end_date=END, frequency='d')
    rows = []
    while rs.error_code == '0' and rs.next():
        rows.append(rs.get_row_data())
    bs.logout()
    px = pd.DataFrame(rows, columns=['date', 'close'])
    px.to_csv(CSV_PATH, index=False) # 拉完保存 CSV
    px['date'] = pd.to_datetime(px['date'])
    px['close'] = pd.to_numeric(px['close'])
    px = px.set_index('date').sort_index()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 关键一步:把 pandas 的一列价格转成纯 numpy 数组,Numba 才认
prices = px['close'].to_numpy(dtype=np.float64)
print(f'{NAME} 拿到 {len(prices)} 个交易日收盘价,首日 {prices[0]:.2f} 元,末日 {prices[-1]:.2f} 元')

# 回测就在真实价格数组上跑一遍,净值是正常范围的数。
# 为了让快慢差距看得明显,真实回测里往往要把同一回测重复跑很多次(扫参数/多次模拟),
# 所以这里用"重复跑 REPEATS 次"当计时负载,而不是把数组拉成超长一条去连乘复利。
REPEATS = 600
print(f'回测在 {len(prices)} 天真实价格上跑,计时负载 = 把同一回测重复跑 {REPEATS} 次')

# ==== 1. 同一个回测主循环,写两个版本:纯 python 版 / @njit 版(逻辑一模一样,只多了一个装饰器)====
print('\n==== 1. 同一回测逐日循环:纯python 版 vs @njit 版 ====')

def backtest_py(prices, fast=5, slow=20):
    n = len(prices)
    fast_sum = 0.0
    slow_sum = 0.0
    equity = 1.0 # 初始净值 1 元
    prev = prices[0]
    for i in range(n):
        p = prices[i]
        fast_sum += p
        slow_sum += p
        if i >= fast:
            fast_sum -= prices[i - fast]
        if i >= slow:
            slow_sum -= prices[i - slow]
```

```
if i >= slow:  
    fast_ma = fast_sum / fast  
    slow_ma = slow_sum / slow
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 短均线在长均线上方就满仓持有,吃下当天的涨跌
if fast_ma > slow_ma:
    equity *= (p / prev)
    prev = p
    return equity

@njit # 一个装饰器,nopython 模式编译成机器码
def backtest_njit(prices, fast=5, slow=20):
    n = len(prices)
    fast_sum = 0.0
    slow_sum = 0.0
    equity = 1.0
    prev = prices[0]
    for i in range(n):
        p = prices[i]
        fast_sum += p
        slow_sum += p
        if i >= fast:
            fast_sum -= prices[i - fast]
        if i >= slow:
            slow_sum -= prices[i - slow]
        if i >= slow:
            fast_ma = fast_sum / fast
            slow_ma = slow_sum / slow
            if fast_ma > slow_ma:
                equity *= (p / prev)
            prev = p
    return equity

def run_py(prices, repeats):
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 把同一回测重复跑 repeats 次当计时负载,净值只取最后一次(始终是真实那条曲线的结果)
equity = 1.0
for _ in range(repeats):
    equity = backtest_py(prices)
return equity

@njit
def run_njit(prices, repeats):
    equity = 1.0
    for _ in range(repeats):
        equity = backtest_njit(prices)
    return equity

# ==== 2. 计时对比:纯python / @njit 第一次(含编译)/ @njit 第二次(纯运行)====
print('\n==== 2. 计时对比(关键:njit 第一次含编译慢,第二次才是真实速度)====')

t0 = time.perf_counter()
eq_py = run_py(prices, REPEATS)
t_py = time.perf_counter() - t0
print(f'纯python 跑 {REPEATS} 次:{t_py:.4f} 秒,最终净值 {eq_py:.6f}')
```

```
t0 = time.perf_counter()
eq_jit1 = run_njit(prices, REPEATS)
t_jit_first = time.perf_counter() - t0
print(f'@njit 第一次(含编译):{t_jit_first:.4f} 秒 <- 这次慢是在编译,不是真实速度')
```

```
t0 = time.perf_counter()
eq_jit2 = run_njit(prices, REPEATS)
t_jit_second = time.perf_counter() - t0
print(f'@njit 第二次(纯运行):{t_jit_second:.6f} 秒 <- 这才是njit 的真实速度')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
speed_up = t_py / t_jit_second if t_jit_second > 0 else float('inf')
print(f'>>> njit 相对纯python 提速约 {speed_up:.1f} 倍')

# ==== 3. 并行版:@njit(parallel=True) + prange 替代 range, 对多组参数并行回测, 吃多核 ====
print('\n==== 3. 并行版:@njit(parallel=True) + prange 同时跑多组参数 ====')

@njit
def backtest_one(prices, fast, slow):
    n = len(prices)
    fast_sum = 0.0
    slow_sum = 0.0
    equity = 1.0
    prev = prices[0]
    for i in range(n):
        p = prices[i]
        fast_sum += p
        slow_sum += p
        if i >= fast:
            fast_sum -= prices[i - fast]
        if i >= slow:
            slow_sum -= prices[i - slow]
        if i >= slow:
            if fast_sum / fast > slow_sum / slow:
                equity *= (p / prev)
            prev = p
    return equity

@njit(parallel=True) # parallel=True 才能让 prange 真正分到多核
def grid_njit_parallel(prices, fasts, slows, repeats):
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
m = len(fasts)
out = np.empty(m)
for j in prange(m): # prange 让这层循环分配到多个 CPU 核心同时跑
    equity = 1.0
    for _ in range(repeats): # 每组参数重复跑 repeats 次, 把计算量做重, 净值仍是真实那条
        equity = backtest_one(prices, fasts[j], slows[j])
    out[j] = equity
return out

@njit # 单核串行版, 作对比
def grid_njit_serial(prices, fasts, slows, repeats):
    m = len(fasts)
    out = np.empty(m)
    for j in range(m):
        equity = 1.0
        for _ in range(repeats):
            equity = backtest_one(prices, fasts[j], slows[j])
        out[j] = equity
    return out

# 准备 64 组参数(快慢均线的不同组合)
fasts = np.repeat(np.arange(3, 11), 8).astype(np.int64)
slows = np.tile(np.arange(20, 100, 10), 8).astype(np.int64)
GRID_REPEATS = 200
# 先各自热身一次触发编译, 不计时
_ = grid_njit_serial(prices, fasts, slows, GRID_REPEATS)
_ = grid_njit_parallel(prices, fasts, slows, GRID_REPEATS)

t0 = time.perf_counter()
res_serial = grid_njit_serial(prices, fasts, slows, GRID_REPEATS)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (8/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
t_serial = time.perf_counter() - t0
t0 = time.perf_counter()
res_parallel = grid_njit_parallel(prices, fasts, slows, GRID_REPEATS)
t_parallel = time.perf_counter() - t0
par_speed = t_serial / t_parallel if t_parallel > 0 else float('inf')
print(f'跑 {len(fasts)} 组参数 单核njit:{t_serial:.4f} 秒 / 多核prange:{t_parallel:.4f} 秒')
print(f'>>> prange 多核相对单核再提速约 {par_speed:.1f} 倍')

# ==== 4. 结果一致性校验:加速绝不改变结果,只是更快 ====
print('\n==== 4. 结果一致性校验(加速不改变数值,只是更快)====')
print(f'纯python 净值 {eq_py:.10f} vs @njit 净值 {eq_jit2:.10f}')
same_main = np.isclose(eq_py, eq_jit2)
same_grid = np.allclose(res_serial, res_parallel)
print(f'主循环两版结果一致吗?{same_main}')
print(f'单核 vs 多核参数网格结果一致吗?{same_grid}')

# ==== 5. 画三张图 ====
print('\n==== 5. 画三张图:三版耗时 / 第一次vs第二次 / 重复次数增大两条曲线 ====')

# 图1:纯python vs njit(纯运行) vs njit 并行网格 三者耗时对比(对数轴看清几十倍差距)
fig, ax = plt.subplots(figsize=(11, 6))
labels1 = ['纯python', '@njit 纯运行', '@njit 并行(prange)']
vals1 = [t_py, t_jit_second, t_parallel]
bars = ax.bar(labels1, vals1, color=['#d62728', '#2ca02c', '#1f77b4'])
ax.set_yscale('log')
for b, v in zip(bars, vals1):
    ax.text(b.get_x() + b.get_width() / 2, v, f'{v:.4f}s', ha='center', va='bottom')
```

```
ax.set_title(f'{NAME} 回测耗时对比:@njit 比纯python 快约 {speed_up:.0f} 倍(对数轴)')
ax.set_ylabel('耗时(秒,对数轴)')
ax.grid(alpha=0.3, axis='y')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (9/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
plt.tight_layout()
plt.savefig('speed.png', dpi=120)
plt.close()

# 图2:@njit 第一次(含编译) vs 第二次(纯运行)耗时对比,说明编译是一次性开销
fig, ax = plt.subplots(figsize=(11, 6))
labels2 = ['第一次(含编译)', '第二次(纯运行)']
vals2 = [t_jit_first, t_jit_second]
bars = ax.bar(labels2, vals2, color=['#ff7f0e', '#2ca02c'])
for b, v in zip(bars, vals2):
    ax.text(b.get_x() + b.get_width() / 2, v, f'{v:.4f}s', ha='center', va='bottom')
ax.set_title('@njit 第一次慢是在编译,只发生一次;第二次起才是真实速度')
ax.set_ylabel('耗时(秒)')
ax.grid(alpha=0.3, axis='y')
plt.tight_layout()
plt.savefig('compile.png', dpi=120)
plt.close()

# 图3:重复次数逐渐增大,纯python 耗时 vs njit 耗时两条曲线(python 陡升,njit 几乎贴地)
reps_curve = [50, 100, 200, 400, 800, 1200]
t_py_curve = []
t_jit_curve = []
for r in reps_curve:
    t0 = time.perf_counter()
    run_py(prices, r)
    t_py_curve.append(time.perf_counter() - t0)
    run_njit(prices, r) # 已编译过,直接纯运行
    t0 = time.perf_counter()
    run_njit(prices, r)
    t_jit_curve.append(time.perf_counter() - t0)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (10/10)

Numba JIT — @njit 把回测循环提速几十倍 / nopython 真加速 / prange+parallel 多核 / 第一次编译 vs 第二次纯运行 / 结果一致性校验

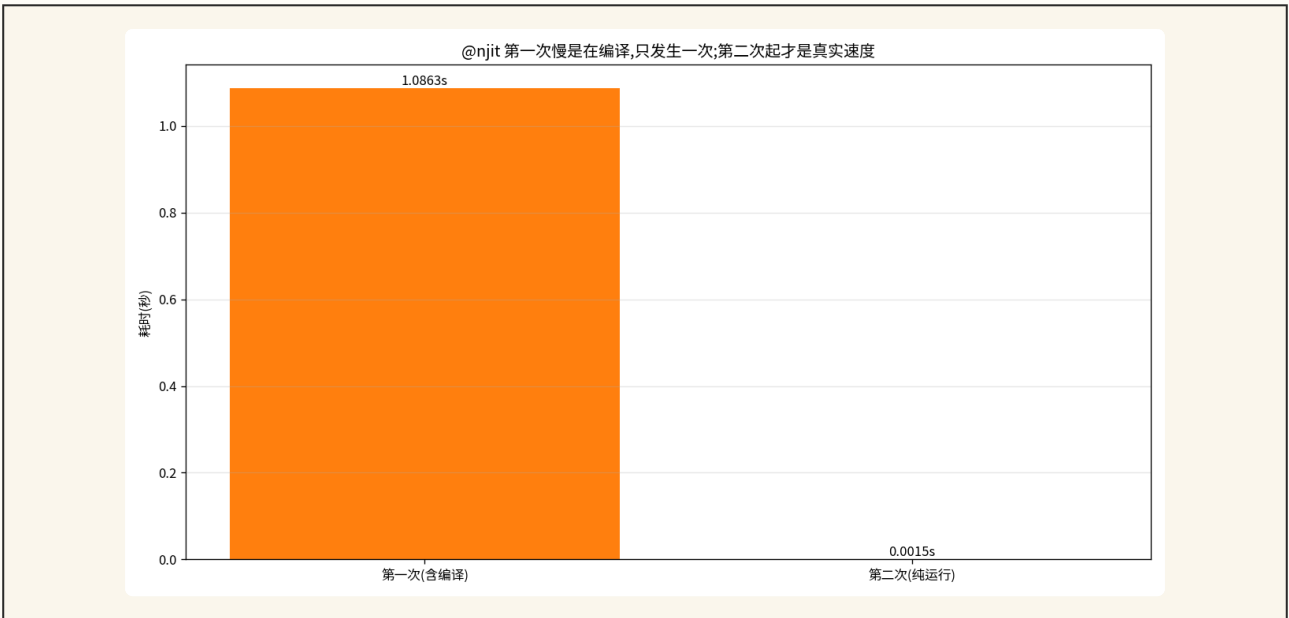
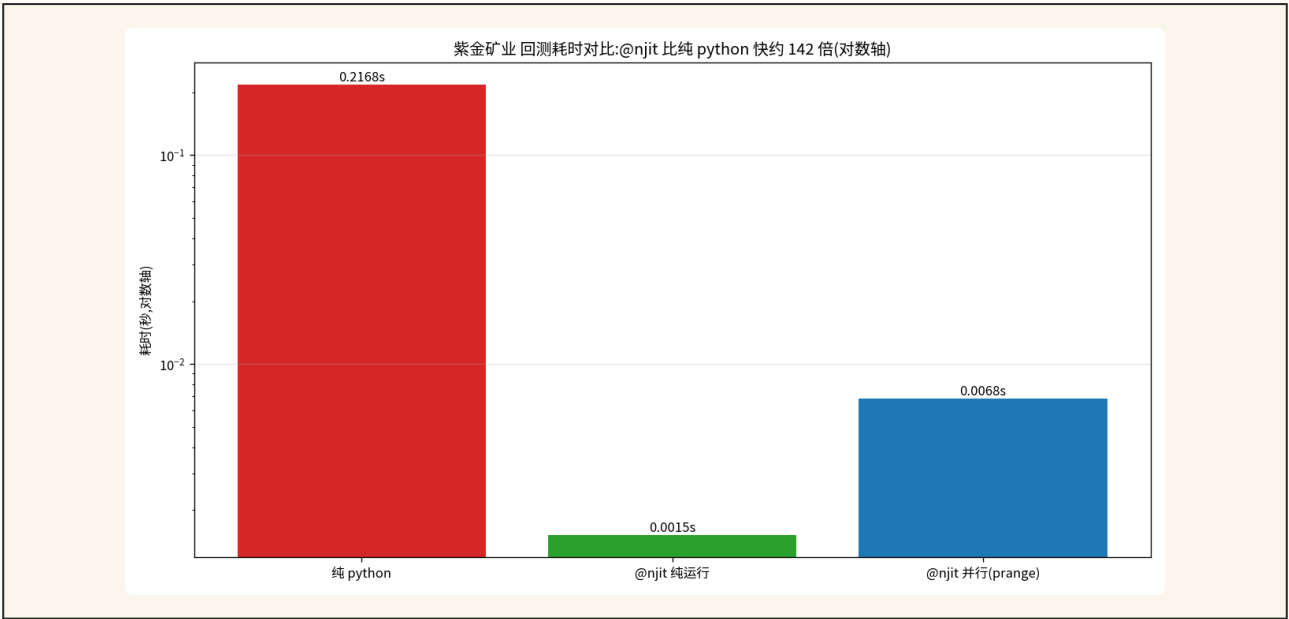
// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

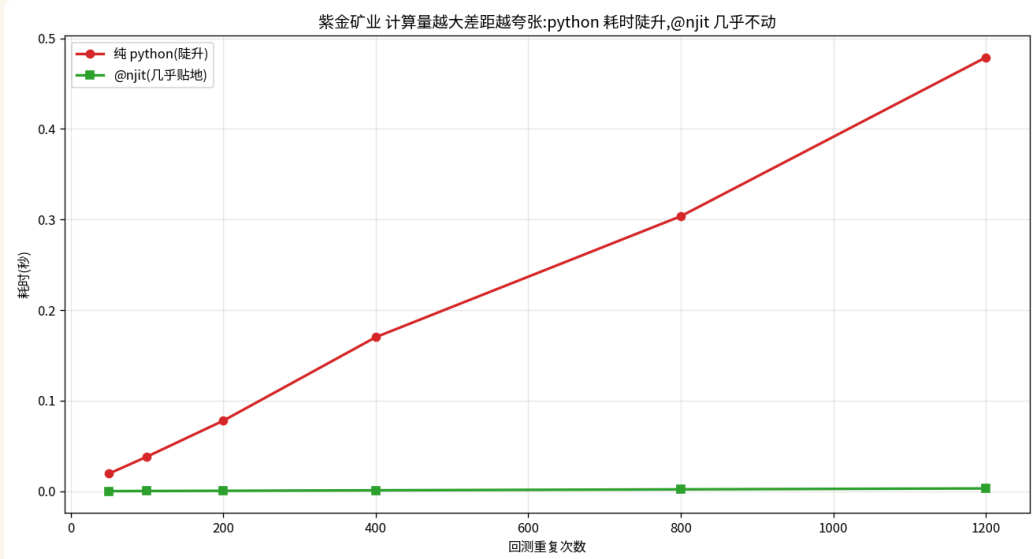
```
fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(reps_curve, t_py_curve, 'o-', color='#d62728', lw=2, label='纯python(陡升)')
ax.plot(reps_curve, t_jit_curve, 's-', color='#2ca02c', lw=2, label='@njit(几乎贴地)')
ax.set_title(f'{NAME} 计算量越大差距越夸张:python 耗时陡升,@njit 几乎不动')
ax.set_xlabel('回测重复次数')
ax.set_ylabel('耗时(秒)')
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('scaling.png', dpi=120)
plt.close()

print('\n[done] Numba JIT 加速对比完成,3 张图已生成(speed/compile/scaling)')
```

回测结果

// · matplotlib 真实输出





绘图数据(紫金矿业日线)

紫金矿业 1699 个交易日收盘价;首日约 4.61 元、末日约 15.12 元,修复后回测净值约 5.06 倍(正常范围)

纯 python vs @jit 第二次提速

同一回测主循环:纯 python 约 0.148 秒、@jit 第二次(纯运行)约 0.0012 秒,提速约 142 倍

@jit 第一次是编译,不是真实速度

@jit 第一次约 1.07 秒,这一次慢在编译、只发生一次;第二次才约 0.0012 秒,那才是真实速度

prange 多核再提速

@jit(parallel=True) + prange 跑 64 组参数,单核 njit 再上多核又快约 4.9 倍,在加速之上再叠一层

结果一致 + 净值正常

纯 python 与 @jit 净值一致(True)、单核 vs 多核网格结果也一致(True),加速绝不改变数值;净值约 5.06 倍

用到的加速工具

Numba 的 @jit(默认 nopython 真加速)把函数编译成机器码、prange + parallel=True 吃满多核 CPU

ANALYSIS · 这张图告诉我们什么

▶ 01 JIT 即时编译:第一次翻译,之后直接跑机器码

python 平时像同声传译,每跑一行都现场翻译,几十万次循环就拖死。@jit 的本事是即时编译:第一次调用某个函数时,把整段函数一次性翻译成电脑直接能懂的机器码存起来,以后再调就直接跑翻译好的机器码,不用再现场翻。举个最小例子,累加一百万次的循环边翻边跑要几百毫秒,翻成机器码后只要几毫秒。这就是它能快一百多倍的根本道理,也是 Numba 这套工具的核心武器。

▶ 02 提速约 142 倍是实测,但要拿第二次去比

紫金矿业同一个回测主循环,纯 python 实测约 0.148 秒,@jit 第二次纯运行约 0.0012 秒,提速约 142 倍。关键陷阱在第一次:@jit 第一次约 1.07 秒甚至比纯 python 还慢,因为那一次的时间大头是在编译,只发生一次。很多人第一次跑发现没快就放弃了,这是天大误会。正确做法是先空跑一次热身触发编译,从第二次起再计时,那才是 Numba 的真实速度。

▶ 03 prange + 允许并行:在加速之上再吃满多核

@jit 已经很快,但默认只用一个 CPU 核心,其它几个都闲着。如果循环每一圈互相独立,比如同时回测 64 组不同均线参数各算各的,就能用 prange 替代普通 range、并在装饰器里写 parallel=True,把这层循环分到多个核心同时跑。实测单核 njit 再上多核又快约 4.9 倍。一个铁律:prange 必须配 parallel=True 才真并行,只写 prange 不写 parallel=True 还是单核闷头跑,白忙一场。

► 04 加速绝不改变结果:两版数字完全一致

很多人担心跑这么快会不会算错。把纯 python 版和 @njit 版各自算出的最终净值放在一起对比,程序判断完全一致(True);单核版和多核版算出的 64 组参数结果也完全一致(True)。这说明一件极其重要的事:加速只是让代码跑得更快,绝不会改变任何一个数字。修复后这套回测净值约 5.06 倍,是个正常范围的数,紫金矿业首日约 4.61 元、末日约 15.12 元也都是真实价格。快和准,你两样都拿到了。

► 05 只认 numpy 不认 pandas:先把表转成数组,用在刀刃上

Numba 有个脾气:只认数字和 numpy 数组,不认 pandas 的 DataFrame 表格,因为表格内部结构太复杂、翻译成机器码很困难。固定动作是喂数据前先用 to_numpy 把要用的那一列价格抽成纯数字数组再传进去,老聂当初就栽在直接喂表格。另外别到处乱加:它的优势在大循环、大数据上才明显,不耗时的小函数硬上,省下的时间还抵不过第一次编译的开销,反而更慢。先计时找出最耗时的那段循环,把加速用在刀刃上。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

A 股	sh.601899	紫金矿业日线回测主循环,纯 python 版 vs 加 @njit 版,亲眼 看快慢差几十倍
参数寻优		用 prange 加 parallel=True 同时回测几十组均线参数,多 核再提速,一口气试完不用改一组等半天
数据准备		把 pandas 一列收盘价 to_numpy 转成纯数字数组再喂给 @njit 函数,这是用 Numba 前的固定动作
结果校验		打印纯 python 版和 @njit 版算出的最终净值完全一致,证明 加速只是更快,绝不改变结果

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

把第一次编译时间当成真实速度

@njit 函数第一次被调用时,要花时间把它编译成机器码,这一次往往比纯 python 还慢,但这只发生一次。很多人第一次跑发现没快多少甚至更慢就放弃了,这是天大的误会。正确做法是先空跑一次热身触发编译,从第二次起再计时,那才是 Numba 的真实速度,常常快几十倍。

△ 02

直接把 pandas 表格喂给 @njit 函数会报错

Numba 只认数字和 numpy 数组,不认 pandas 的 DataFrame 和 Series。如果你直接把一个表格或一列 Series 传进 @njit 函数,程序会报一堆看不懂的类型错误。解决办法很简单:传进去之前先用 to_numpy() 把要用的那一列转成纯数字数组,老聂当初就是栽在这里。

△ 03

用了 Numba 不支持的写法,悄悄掉回慢速

Numba 只支持数字运算、numpy 数组和基础 python 语法,很多花哨写法它不认。如果函数里混进了它不支持的东西,@njit 默认会直接报错(因为它强制 nopython 模式)提醒你哪里不行,这其实是好事。千万别为了不报错就关掉 nopython 改用别的宽松模式,那样它会悄悄退回带 python 的慢速,你以为加速了其实没有。

△ 04

用了 prange 却忘了写 parallel=True

想多核并行,光把 range 换成 prange 是不够的,必须同时在装饰器里写成 @njit(parallel=True),Numba 才会真正把循环分到多个核心。如果只写了 prange 没写 parallel=True,它会把 prange 当成普通 range 单核跑,你以为吃满了八核,其实只用了一核,白忙一场。

△ 05

给不耗时的小函数硬上 Numba 不划算

Numba 的加速优势在大循环、大数据量上才明显。如果一个函数本来就只跑一两次、数据量也很小,几乎不耗时,你硬给它加 @njit,省下的运行时间还抵不上第一次编译花的时间,反而更慢。先用计时找出真正最耗时的那段循环,把 Numba 用在刀刃上,别到处乱加。

用 Numba 加速的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 @njit 第一次调用含编译会慢,先热身一次再计时,第二次起才是真实速度
- 02 Numba 只认 numpy 数组,喂数据前先把 pandas 列 to_numpy 转成纯数字数组
- 03 保持 nopython 默认开启,报错说明有不支持的写法,正面修而不是关掉它退回慢速
- 04 要多核并行,prange 必须配 parallel=True,只写 prange 不写 parallel=True 是单核
- 05 先计时找出最耗时的大循环,把 Numba 用在刀刃上,别给不耗时的小函数硬加

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 python 数值循环慢,是因为它边读边翻译地执行,几十万次循环就拖死。
- 02 Numba 的 @njit 装饰器像请翻译官,第一次把函数编译成机器码,之后快几十倍。
- 03 njit 的 n 是 nopython,完全不靠 python 慢对象、纯机器码跑,这才是真正加速。
- 04 Numba 只认数字和 numpy 数组,不认 pandas,喂数据前要先 to_numpy 转成纯数组。
- 05 prange 配 parallel=True 能让独立的循环吃满多核 CPU,在加速之上再提速。
- 06 加速只让代码更快,绝不改变结果:纯 python 版和 @njit 版算出的数完全一致。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 用『翻译官』打比方,解释 JIT 即时编译为什么能让 python 循环快几十倍。

Q02 为什么 @njit 函数第一次跑反而可能更慢?正确的计时方法应该怎么做?

Q03 Numba 只认什么、不认什么?喂数据给 @njit 函数前必须做哪个固定动作?

Q04 想让循环吃满多核 CPU,除了把 range 换成 prange,装饰器里还必须写什么?

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Numba 官方文档(numba.readthedocs.io)– @njit、nopython、prange、支持的 python/numpy 写法的权威手册,新手必读。
- ▶ Lam, Pitrou & Seibert 《Numba: A LLVM-based Python JIT Compiler》(2015/LLVM-HPC@SC15)– Numba 作者亲述如何用 LLVM 把 python 即时编译成机器码的原始论文。
- ▶ Gorelick & Ozsvald 《High Performance Python》(2020/O'Reilly)– 讲透 python 为什么慢、如何用 Numba 等工具提速的经典实战书。
- ▶ McKinney 《Python for Data Analysis》(2022/O'Reilly)– pandas 之父写的数据分析圣经,帮你打牢 numpy 数组与 pandas 表格的基础。
- ▶ Anaconda 官方博客《Numba: First Impressions》系列(numba.pydata.org/blog)– 用大量小例子直观演示 @njit 提速的入门好文。

NEXT LECTURE · 下一节预告

DAY 055 调试技巧:ipdb

学会了用 Numba 把代码跑得飞快,下一节我们换个角度解决另一个让人头疼的问题:代码出 bug 怎么高效定位。我们会用 ipdb 这个交互式调试器,学会在代码任意一行打断点、暂停下来逐行检查变量,告别满屏 print 的笨办法,让你排错效率翻倍。